
Latent Memory-Augmented LLMs

Matthew Busenleener
mbusenle@usc.edu

Abstract

We present a memory-augmented language model inspired by how humans integrate long-term memory into ongoing reasoning. Human cognition relies on continual interaction between working memory and episodic recall, dynamically retrieving and integrating prior experiences to guide problem solving. Our model brings this principle to large language models by conditioning reasoning on task-optimized latent memories stored from prior forward passes. Each transformer layer maintains a pool of latent memories that is selectively observed by the current hidden states via cross-attention. During memory encoding, hidden states at each layer are compressed into K memory vectors and appended to the pool. Memory-attention during decoding scales linearly in complexity with respect to the number of memories in the pool, significantly reducing inference costs over long contexts. We evaluate our model on reading comprehension using the HotpotQA dataset. Our experimental results show that the latent memory-augmented model outperforms the In-Context baseline by 22% while compressing contexts by 8x.

1 Background

Large language models (LLMs) exhibit impressive performance on a variety of tasks, but their adaptability is constrained by their reliance on static, parameterized knowledge learned during training. Furthermore, domain/task-specific finetuning is hindered by the coupling of knowledge and reasoning in the weights of LLMs. This entanglement often results in catastrophic forgetting, a phenomenon in which an LLM forgets previously learned skills or knowledge during finetuning. Due to these challenges, there is a growing interest in integrating human-inspired long-term memory retrieval systems into LLMs. The most widely used approach is Retrieval-Augmented Generation (RAG) [1] [2], in which a search agent retrieves relevant external text documents that are inserted with the prompt into the LLM's context window. While RAG improves accuracy and adaptability, its reliance on fitting additional information into the model's input context window has various limitations. As the context window grows in length, the LLM's attention computations scale quadratically, making long contexts inefficient and difficult to model. Furthermore, a one-time retrieval of raw text differs significantly from human memory, which operates through dynamic, recurrent interactions between working memory and long-term memory.

Recent research is increasingly focused on enhancing LLMs with latent-space memory, where external information is stored as compressed hidden-state representations rather than raw text. In this approach, the model can access latent "memories" of previous facts or experiences within its own hidden-state space, allowing more flexible, dynamic, and human-like interactions with memory. There are several key advantages of latent-space memory. For one, these methods typically utilize some method of context compression, reducing storage overhead and improving the efficiency of future processing. Another advantage is that latent-space memory can be optimized in end-to-end gradient-based learning, allowing for task-optimized memory encoding and utilization. Finally, the notion of encoding knowledge into latent representations strongly aligns with human memory systems, where information is stored through distributed neural activations. Implementations such as M+ [3] and MemoryLLM [4] maintain a memory pool of hidden activations from prior inputs

that are retrieved during inference via pretrained similarity functions and exposed to the LLM’s current hidden states via cross-attention. Other implementations such as Larimar [5] equip LLMs with episodic memory retrieval by training them with read/write triggers that adaptively retrieve/store latent memories when needed. Much like humans, these architectures continuously integrate and update contextual knowledge through selective memory recall during problem solving.

2 Methods

We propose a memory-augmented language model that integrates task-optimized latent-memory attention into the intermediate reasoning states of the LLM during inference. Each transformer layer maintains a pool of latent memory tokens derived from cached compressed hidden states at that layer from previously read/encoded documents.

When encoding a document into the memory pool M , we first divide the document into chunks roughly of size c , splitting at the nearest sentence boundary. We then independently pass each chunk through the LLM and extract the last K hidden states at each layer l as a compressed intermediate representation of the text chunk and insert them into M_l . In a decoder-only LLM with a causal mask, the last token-level hidden states are a suitable compressed representation of the LLM’s current state since they have computed attention across all previous token-level hidden states. By chunking the documents, the complexity of encoding a document of length n using the LLM is reduced from $O(n^2)$ to $O(c * n)$, where $c \ll n$. The compression rate by taking the last K hidden states is c/K . In our experiments, we set c to 128 and K to 16, achieving 8x compression of documents with an encoding procedure that scales linearly with respect to document length.

During decoding, we employ a TCM-style iterative update at each intermediate layer of the LLM. Given the current sequence of n hidden states at layer l of the LLM, $h_{l,[1:n]}$, we prepend the flattened memory pool M_l to produce a sequence of embeddings of length $(m * K) + n$ where m is the number of memories ingested into the memory pool. In the next self-attention layer in the LLM, we apply attention masking to exclude the memory tokens from the query vector such that attention is computed from h_l onto $[M_l, h_l]$. As such the complexity of this procedure is $O(m * K + n)$, escaping the traditionally quadratic complexity of self-attention while allowing us to take advantage of the LLM’s pretrained attention weights. At each layer’s update step, we replace the memory tokens from the prior layer with the current layer’s memory pool. Through these methods, we allow integration of stored latent memories into the current hidden states through cross-attention at every layer.

To train this model for memory encoding and memory-conditioned decoding, we maintain two independent LoRA adapters over the same base model (one for encoding, one for decoding) that are trained across a pretraining stage followed by a QA finetuning stage. LoRA adapters (Low-Rank Adaptation) are a Parameter-Efficient Fine-Tuning (PEFT) method where we add small trainable matrices modeled through low-rank decomposition that are summed with the original LLM weight matrices during inference. By training only these lightweight adapters and keeping the base LM frozen, we can tune LLM behaviors much more efficiently. These two levels of modularity are designed to address the difficulty in training this system end-to-end. The decoder LoRA adapter can only learn to properly utilize latent memories to answer questions if the encoder already produces stable, informative memory representations. Similarly the encoder will only receive meaningful learning signal if the decoder knows how to properly attend to and exploit the encoded memories. These learning dependencies introduce a "cold-start" problem that creates unstable optimization dynamics when both components are trained jointly from scratch on question answering. Furthermore, our QA from context training task is a difficult task that the base model with the full context performs relatively poorly on. As such, for very difficult samples the learning signal to the encoder is very weak since simple understanding the context is not sufficient for answering the question correctly. So, we first stabilize the encoder-decoder interface using a memory reconstruction objective before finetuning for QA.

In all experiments, we use Qwen2.5-1.5B as our pretrained decoder-only base LLM. All the base model parameters are frozen throughout training. We introduce two separate LoRA adapters, one used during context encoding and one used during decoding. Specifically, for each transformer block we attach low-rank adapters to the attention projections W_q, W_k, W_v, W_o and the MLP projections $W_{gate}, W_{up}, W_{down}$. The encoder adapter θ_{enc} and decoder adapter θ_{dec} share the same LoRA hyper-

parameters (rank $r = 16$, scaling $\alpha = 32$, dropout 0.05), but maintain distinct parameters and are activated exclusively in their respective phases.

In addition to LoRA, we learn two single-token soft prompts: an autoencoding soft prompt $p_{\text{AE}} \in \mathbb{R}^{1 \times d}$ and a finetuning soft prompt $p_{\text{FT}} \in \mathbb{R}^{1 \times d}$, where d is the hidden size of the base model. These prompts are prepended to the decoder input during reconstruction and QA finetuning respectively. The set of trainable parameters is thus

$$\Theta = \{\theta_{\text{enc}}, \theta_{\text{dec}}, p_{\text{AE}}, p_{\text{FT}}\}.$$

All gradients are restricted to Θ .

We train and evaluate on a dataset of 7500 samples from HotpotQA fullwiki. Each training example consists of a question q , a short answer a , and a set of independent Wikipedia passages where some passages are relevant to the question and some are not. HotpotQA is specifically a multi-hop QA dataset where success depends on reasoning across otherwise independent documents/passages. We include irrelevant passages to ensure the model is robust to task-irrelevant memories and must learn to modulate attention to find the most relevant memories. Each passage is further divided into chunks according to the methods discussed above. Each example therefore yields a list of contexts $\mathcal{C} = c_1, \dots, c_m$. At training time, contexts are ingested into memory once per example using the encoder pathway, and the question is answered by the decoder pathway conditioned on those memories.

In the first stage, we pretrain the memory encoder and memory-conditioned decoder with an autoencoding objective for one epoch. The goal is to ensure that the encoder learns to write compressive yet information-preserving memories, and that the decoder learns to read from these memories to reconstruct their source text. For each context $c_i \in \mathcal{C}$, we activate the encoder adapter θ_{enc} and run a forward pass. For each layer $l \in 1, \dots, L$, we extract the last K hidden states

$$M_l(c_i) = h_{l, [|c_i| - K + 1 : |c_i|]} \in \mathbb{R}^{K \times d}.$$

and store them as the memory tokens for that context. In reconstruction pretraining, each context is encoded independently with no other contexts included in the pool. We then reconstruct c_i from its memories. We activate the decoder adapter θ_{dec} and form the decoder input sequence

$$x_{\text{AE}} = [p_{\text{AE}}, \text{"Answer:"}, c_i, \text{eos}].$$

tokenized using the base tokenizer. The memory pools $M_l(c_i)_{l=1}^L$ are injected into every layer via the cross-attention masking described above. Reconstruction therefore requires the decoder to attend to $M_l(c_i)$ at each layer to predict the full context tokens.

We apply a standard next-token cross-entropy loss. To prevent the model from learning to auto-complete the fixed prefix, we mask the soft prompt and the literal prefix "Answer:" from the loss. Let y_{AE} be the tokenized target sequence corresponding to x_{AE} . We set the labels for the prefix positions to -100 , so gradients only flow through context tokens. Therefore, the reconstruction loss is

$$\mathcal{L}_{\text{AE}} = - \sum_{t \in \text{ctx}} \log p_{\Theta}(y_t \mid y_{<t}, M(c_i)).$$

During this stage, gradients arise from reconstructing the context tokens. These gradients flow back through (i) the decoder adapter θ_{dec} and (ii) the memory-attention pathway into the encoder adapter θ_{enc} , since the memories $M_l(c_i)$ are differentiable functions of the encoder forward pass. The base model weights remain frozen and receive no gradients. The AE soft prompt p_{AE} is updated jointly with both adapters. Through this pretraining objective, the encoder learns to produce compressed memories that capture all the original information while the decoder learns to understand the latent memories in isolation.

After reconstruction pretraining, we begin the training over the QA objective for subsequent epochs. This stage teaches the decoder to use a pool of multiple memories to answer questions and teaches the encoder to store task-relevant information rather than purely reconstructive information. For each example, we encode all contexts $\mathcal{C} = c_1, \dots, c_m$ using the encoder adapter θ_{enc} exactly as in Stage 0. Memory tokens from all contexts are concatenated per layer:

$$M_l(\mathcal{C}) = \text{concat}(M_l(c_1), \dots, M_l(c_m)) \in \mathbb{R}^{(mK) \times d}.$$

We activate the decoder adapter θ_{dec} and pass the soft prompt and question through the decoder. As in reconstruction, memory pools are injected at every layer. The decoder must attend to $M_l(\mathcal{C})$ to generate the answer a . We use next-token cross entropy, masking the finetuning soft prompt and the question/prefix tokens so that the loss applies only to the answer:

$$\mathcal{L}_{\text{QA}} = - \sum_{t \in \text{ans}} \log p_{\Theta}(y_t \mid y_{<t}, M(\mathcal{C})).$$

Gradients from $\mathcal{L}_{\text{QA}}$ update the decoder adapter θ_{dec} and finetuning soft prompt p_{FT} . Because answer prediction depends on memory attention, gradients also backpropagate into encoder adapter θ_{enc} , shaping the stored latent memories toward QA-relevant compression. This provides end-to-end task optimization of the stable encoder–decoder setup established in during pretraining.

Training is performed with batch size 1 due to memory overhead of inference and to simplify early issues related to padding masks across larger batches. We use gradient accumulation with 16 steps, yielding an effective batch size of 16. Parameters in Θ are optimized with AdamW (learning rate 10^{-4} , no weight decay). We clip gradient norms to 1.0 at each optimizer step. Validation loss is computed after every epoch using the QA objective, and early stopping with patience 1 and minimum delta 10^{-4} is applied. The best checkpoint stores both LoRA adapters and the two soft prompts. Overall, the curriculum can be summarized as:

1. **Reconstruction pretraining (epoch 0).** Encode each context independently into latent memory tokens using θ_{enc} . Decode and reconstruct the context using cross-attention over memories and update θ_{enc} , θ_{dec} , and p_{AE} under $\mathcal{L}_{\text{AE}}$.
2. **QA finetuning (epochs 1+).** Encode all contexts jointly into a per-layer pooled memory. Decode answers conditioned on the memory pool and update θ_{enc} , θ_{dec} , and p_{FT} under $\mathcal{L}_{\text{QA}}$.

This two-stage procedure stabilizes learning by first ensuring that the memory channel carries faithful information and that the decoder can read from it, after which QA finetuning can successfully shape memory contents toward task-specific retrieval and reasoning.

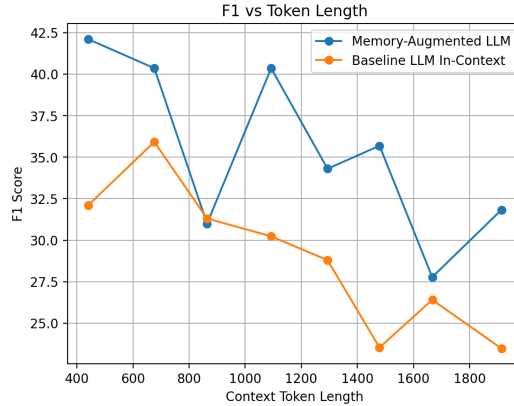


Figure 1:

3 Findings

We evaluated the latent memory-augmented model against a strong in-context baseline using the same underlying base architecture (Qwen2.5-1.5B). The baseline model prompted with the full concatenated contexts, while our memory-augmented model ingests the same contexts once into its latent memory pool and answers questions by cross-attending to the pooled memories at every layer. To evaluate performance as a function of context length, we gather groups of samples across increasingly large windows of token length from roughly 400 to 1800 tokens from the HotpotQA validation split. For each example we computed a cheap proxy for length: the character length of combined text = question + contexts. We sorted by this string length, trimmed extreme outliers,

and partitioned the remaining examples into 8 bins around evenly spaced length centroids. Each bin contained up to 150 examples, prioritizing samples closest to that bin’s centroid. After filtering, we computed true token lengths only for the selected samples, and used the average token length of each bin as the x-axis value. Both models were evaluated on identical bins, and each produced short free-form answers (max 5 new tokens) under greedy decoding. We report token-overlap F1 using standard normalization (lowercasing, punctuation and article stripping), which is typical for HotpotQA’s one-to-two word ground-truth answers.

As seen in Figure 1, our latent memory-augmented model achieved 35.43% average F1, while the baseline in-context LLM achieved 28.97% average F1, showing a +6.46 absolute F1 improvement across the full evaluation set. Both models showed performance degradation across longer context lengths, losing about 10 F1 points from the shortest contexts to the longest ones, indicating that longer contexts remain challenging even when amortized into latent memory.

Although our results did not show improved long-context robustness, our model processes long contexts substantially more efficiently. Using long-context chunking and encoding, we achieve 8x compression of contexts. Crucially, this compression does not incur the quadratic attention cost that the baseline pays when the full context window grows. In practice, this means that even when accuracy trends are parallel, our model offers a strong efficiency advantage for long-context processing. This efficiency advantage will also allow for training over longer contexts than would be feasible with the baseline LLM in the future, which may result in improved relative performance.

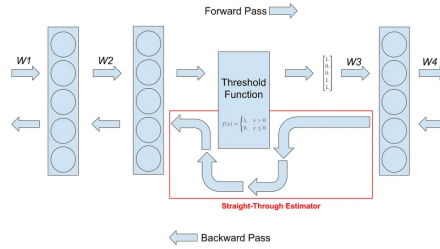


Figure 2:

4 Discussion

This work adds evidence that latent-memory architectures can be a practical and cognitively meaningful alternative to standard in-context prompting. The core idea of compressing contexts into internal memory traces and iteratively attending them during problem-solving mirrors a key property of human reasoning: we do not re-read the world in full detail to solve each new problem. Instead, we rely on earlier-formed, compressed internal representations and re-activate them as needed. In the model, the encoder plays a role analogous to memory consolidation, transforming raw experience into a compact latent trace, while the decoder resembles working-memory reasoning that selectively integrates those traces across layers.

From an AI systems perspective, the approach addresses the quadratic scaling bottleneck of long-context attention. Even though the current experiments did not yet show improved accuracy retention at the longest context lengths, the model already provides an efficiency advantage: contexts are ingested once, compressed by design, and reused at linear cost. This makes the architecture attractive for settings with repeated queries over large, static databases (e.g., personal assistants, scientific note-taking, long-lived agents). In these domains, computational savings are as important as accuracy.

From a computational cognition perspective, this model offers ideas about how memory compression and recurrent recall might support reasoning. Because memory tokens are derived from intermediate hidden states and tied to the model’s internal representational geometry, “remembering” is not just retrieving text but re-activating a distributed latent state. This is closer to human memory than RAG-like paradigms, which are more like perfect external lookup.

An important element of cognition-inspired architectures for memory retrieval is that human memory is not uniformly faithful or accessible. It is compressed, biased, cue-dependent, and reconstructive.

By storing only the last-K hidden states per chunk, the present model imposes a strong inductive bias similar to human recency effects where later-context representations dominate what is stored. Cross-attention then produces a retrieval process that is cue-dependent and reconstructive. This can be beneficial since it encourages the model to store task-relevant summaries rather than verbatim text and to integrate memory with current goals. It also risks incorporating human biases like interference between similar traces and overreliance on recency. In future experiments, we would like to investigate these biases, identifying whether performance is better on contexts where the "answer" is in the last K tokens and whether the memory cross-attention favors similar but irrelevant memories.

There are several limitations of the current approach. For one, storing the last K hidden states is a strong and convenient heuristic, but it may throw away useful information in the beginning of the context that is poorly relayed to the last K tokens, especially in early layers. This likely contributes to the similar decay slopes seen relative to baseline LLMs at longer contexts. The decrease in long-context performance is also likely associated with the limitations of our training paradigm, which provide relatively short and sparse memory pools. As such the model has little incentive to learn strategies for handling very large memory pools.

Secondly, our training/evaluation tasks focus on semantic/factual memory only. This is a very limited paradigm that does not explore many of the ways that humans utilize memory during reasoning. In future experiments, we would like to explore more diverse memory-related use cases.

Our biggest limitation is that our functional memory capacity is small and without true sparse retrieval. Because the full memory pool is observed at every layer, practical memory pool size is limited. Attending to all stored traces scales linearly in pool size, but linear still becomes expensive for very large m . More importantly, indiscriminate recall invites interference: irrelevant memories compete for attention, lowering the signal-to-noise ratio as the pool grows. Humans solve this with selective recall, but our simplified model does not yet.

In early exploration, retrieval was a key direction that we explored. The intended design was cognitively very suitable. At each layer, learned query and key projections would compute relevance between the current hidden state and stored memories, selecting the top-k traces for cross-attention. Such a mechanism resembles human recall, where cues probe memory and only a few relevant episodes enter working memory. The hope was that using retrieval in this way we could scale to arbitrarily large memory pools.

However, implementing this in a differentiable, stable way proved difficult. The bottleneck is that top-k selection is non-differentiable. Under strict top-k masking, retriever weights do not receive gradients because unselected memories cannot influence the downstream loss. One solution that we explored was using a straight-through estimator with Gumbel sampling [6] to make the mask soft in the backward pass, allowing gradients to flow. Shown in Figure 2, with a straight-through estimator we use hard top-k masking over the per-memory retriever scores during the forward pass, but during the backward pass we pretend that the mask was soft so that gradients can flow to the retriever weights. However, these updates are very high variance because we still cannot update the retriever weights with respect to unselected memories. The only learning signal that the downstream loss can contribute to the retrieval weights are with respect to the selected memories. Given that the QA learning signal itself is already weak and dependent on a careful training paradigm, this additional noise prevented stable retriever learning. In practice, the retriever converged to only 50–60% correct context selection, which was not sufficient to support reliable reasoning and in some cases harmed performance.

A second conceptual challenge was mismatch with the pretraining regime. During reconstruction pretraining, every layer observes memories from the same source context, and layerwise retrieval across different contexts is never trained. A retriever that changes its selected memories at each layer is promising, but it requires a training task where such dynamic mixing is meaningful.

Eventually we simplified the paradigm, allowing full observation of the memory pool at the expense of selective recall. This change stabilizes training and yields clear efficiency benefits relative to raw prompting, but it compresses functional capacity because pool size cannot grow indefinitely without interference and compute costs. This simplification can be partially mitigated by coupling the model with traditional RAG to retrieve the indices for relevant documents in the memory pool, but this hybrid drifts away from a human memory model.

There are several directions in which we would like to extend this work. The within-pass co-trained retriever is a key missing piece. Alternatives to straight-through top-k could reduce variance and improve learnability. One task where adaptive memory retrieval that can select unique memories across different layers might be useful for multi-step reasoning problems, where we could encode more general reasoning traces or problems into memory and attend to them when performing functionally related but semantically distinct tasks. This paradigm would simulate experiential or procedural memory, where we could also encode rewards and failures associated with prior reasoning chains.

We would also like to explore lifelong memory pool evolution with selective update mechanisms rather than clearing memory pools in-between tasks. Humans accumulate memories over extended periods, undergoing selective pruning and consolidation of former memories.

Finally, we would like to more thoroughly explore long-context modeling. The central advantage of this architecture of long-context efficiency is currently under-exploited, and future works could evaluate whether latent-memory approaches can increase the functional context window of LLMs and improve long-context retention and reasoning.

In its current form, the model demonstrates that a cognitively motivated latent-memory interface can improve QA accuracy while providing substantial efficiency gains over raw in-context baselines. The absence of improved long-context robustness should be viewed less as a failure of the memory mechanism and more as a signal that memory systems must be trained under longer context conditions to develop strong recall strategies. The early retrieval exploration shows that while selective recall is essential for scalability, it is difficult to learn without careful curricula and low-variance optimization tools. Bridging that gap toward experiential, strategy-level memory offers a rich path for both better AI systems and deeper computational models of human cognition.

References

- [1] Khandelwal et al. (2019) Generalization through Memorization: Nearest Neighbor Language Models. *arXiv preprint arXiv:1911.00172*.
- [2] Zhong et al. (2023) MemoryBank: Enhancing Large Language Models with Long-Term Memory. *arXiv preprint arXiv:2305.10250*.
- [3] Yu Wang et al. (2025) M+: Extending MemoryLLM with Scalable Long-Term Memory. *arXiv preprint arXiv:2502.00592*.
- [4] Yu Wang et al. (2024) MEMORYLLM: Towards Self-Updatable Large Language Models. *arXiv preprint arXiv:2402.04624*.
- [5] Das et al. (2024) Larimar: Large Language Models with Episodic Memory Control. *arXiv preprint arXiv:2403.11901*.
- [6] Siyuan Huang et al. (2025) Gumbel Reranking: Differentiable End-to-End Reranker Optimization. *arXiv preprint arXiv:2502.11116*.
- [7] Yang et al. (2018) HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. *arXiv preprint arXiv:1809.09600*.